

Auto Prompt Evolution

Lanza Luca

May 21, 2026

1 Introduction

This short experiment aims to test limits and capabilities of SLM in solving reasoning problems, and how prompt optimization can improve their performance. The experiment also builds an auto evolver model which takes an incorrect answer and iteratively tries to build a better prompt, to get an improve in the reasoning from the model.

2 The dataset

The problem that is presented here is the capability of small language models of reasoning in order to solve a logical problem. To do that different models from the Qwen family are taken and test on BIG-Bench Dataset, which contains many different problem and logic puzzles. As test folder Five Objects is taken, which contains many different kind of logical sorting puzzles, where five elements must be sorted in a correct way given determined constraints. The file is a json where information is organized as inputs, containing all the constraints useful to solve the logical puzzle, and a target score, which contains 5 rows, one for each object, where only the correct answer is labeled with one, while the other rows get 0 as label. This dataset is chosen because it contains puzzles that are not too complicated for a SML to solve, but can still pose a challenge and force the model to deal with abstract concept and link number and order(like the oldest, the rightmost) that can confuse it in the moment in which it have to represent this concept in an ordered list (for example the rightmost will be the 5th object).

3 The idea of the project

The project is divided in 3 python modules:

- The first module is called data handler and it is responsible for loading the dataset and to draw random sample from it, that will later be given to the model that is being trained. It also divides the questions in two parts: the training set and the evaluation set so that the prompt tested will never receive the same question.
- The second module is called llm client and it has the simple role of loading the model and to manage the prompts that are passed to the various models in the last module.
- The third module is called prompt evolver and it is the actual core of the structure. Inside this module the model that are used in the experiment reason about the problem to solve, it get evaluated by another model and then, the prompt is evolved using as a sample one of the errors made.
- Lastly in colab the modules are run to get the initial prompt and the evolved best prompt, and compared on generic samples taken from the evaluation set to see which one get the best result (hopefully the evolved prompt).

3.1 The reasoning model chosen

The models that was choosen as "reasoner", that carry out the actual reasoning and try to solve the puzzles are 3 member of the Qwen family: Qwen 0.5B, Qwen 1.8B and qwen 7B. While Qwen 0.5B performed really poorly and couldn't really be evaluated, because of the lacking in performance, the other two models test gave interesting results, In particular the 7B parameters model achieved good results in the reasoning when it had a good prompt. The picture below shows how this last model achieved, with the right prompt obtained in the training phase, far superior performance. The model is also able to act both as a judge and even to do prompt evolution, but to avoid sycophancy (where the model convince himself that the reasoning made by himself is correct) it is better to use another model to carry out those tasks.

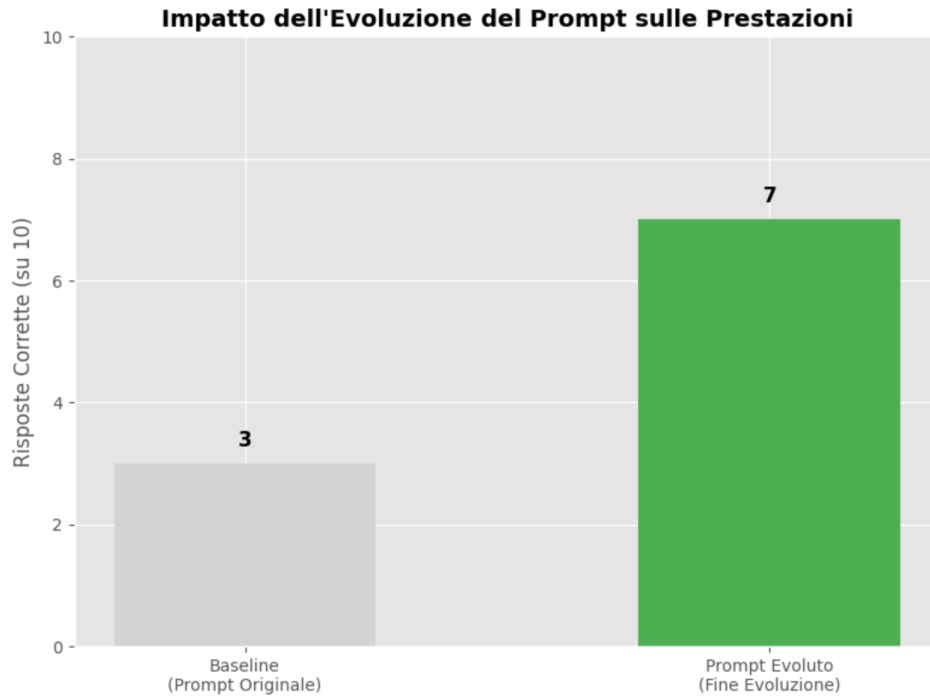


Figure 1: On the left the performance of the Qwen 7B model with the basic prompt, on the right the prompt that achieved the best results in the training.

3.2 The judge and prompt evolver model

To evaluate the answer of the reasoning model another model is used, specifically Gemma 9B, a model with more reasoning capabilities than my tested ones, which make it a good judge with superior reasoning capabilities, able to reason about models logical process and to recognize when it fails. For the same reason it's a good decision to use it as the prompt evolver and so, while the logical solver were tested on multiple small models, the judge and the prompt evolver are only be tested using Gemma so far.

3.3 limit of the models

While the limitation will be discussed better in the specific paragraph, it is important to note that llm as a judge will not be perfect and will not always perfectly capture the correct reasoning and sometimes it will do the opposite, assuming a wrong reasoning correct, leading to some false negative and false positive cases.

4 Algorithm implementation

In this section will be discuss how the algorithm works and how each module works.

4.1 Data Handler

The data handler is intialized by loading the specific folder (using the load data function) that is given in input when the module is istantiated. After that the data is partitioned in training and evaluation set, to avoid data leakage from one phase to the other. After that, for both the two sets a get random sample function is implemented, to get a random example from the correct dataset. Finally, the example that is drawn is formatted to return a structure that contains the question, all the possible options and the actual correct answer that is fed to the judge.

```
{
  "input": "On a shelf, there are five books: a green book, a blue book, a brown book, a gray book, and a red book.",
  "target_scores": {
    "The green book is the third from the left.": 0,
    "The blue book is the third from the left.": 1,
    "The brown book is the third from the left.": 0,
    "The gray book is the third from the left.": 0,
    "The red book is the third from the left.": 0
  }
},
```

Figure 2: An example of how a problem is presented in the BIG-Bench dataset.

4.2 LLM client

The LLM client module first (in the initialization phase) checks using torch if a GPU is available, else it runs on CPU, this is done to get the best speed possible on the hardware. If GPU are available then the precision of the weights is set to float 16, or single precision (float 32) if CPU is used. This is crucial to improve the performance on GPU, where multiplication on half precision are optimized. Finally, the pipeline takes the input, tokenize it, generates the text and converts it in natural language. The function prompt model instead takes the prompt, the max number of tokens allowed for the generation and the temperature (which means that the text generated will be taken from a distribution sample with a certain probability). System prompt is an optional parameter that is used to instruct the model on is task and how it should behaves. The output variable get a value from the self.pipe function call that use pipeline to generate the text, setting the received prompt, the max length of the answer(output) and the temperature, letting the model being more "creative" or more strict.

4.3 Prompt Evolver

The prompt evolver is composed of 3 specific models: The judge, the reasoner and the evolver. In the experiment both the work of judge and evolver are carry out by Gemma, while the reasoner is a model from the Qwen family. The init define the client and the judge that will be used, together with the prompt that has to be fed to the reasoning model to solve the puzzles. The evaluate answer model takes the model answer and the correct answer extracted before, and are passed in an optimized prompt to evaluate the correctness of the reasoning. The prompt in particular used few shot prompt engineering and a set of rule that explain the reasoning step that the judge model (which in the experiment has 9B parameters) has to follow to guide it in evaluating the reasoner answer. In this sense the model has to figure out the final answer from the student (reasoner) model, understand how the student ordered the objects, match this order with the correct answer and find out if the student answer is correct, and conclude with [YES] in the valid case. After that the correctness is simply evaluated by catching the [YES] as positive case and giving a score of +1, 0 otherwise. The structure of the code is think in two phases: a training phase, where you apply a test on batch of questions that are equals for all the prompt evolution, in this way all the prompts are trained on the same questions (in my case 5 questions) and the accuracy is calculated as corrected answer from the student on all the question asked. This evaluation on a single prompt on a batch is what is done in the function evaluate on batch. Run evolution loops n times, which are also the number of new prompts that will be generated.It also receive as inputs on how many question those prompts will be tested (the batch

size) and how big the student model is, to give the option to the prompt evolver to guide the evolution in two different ways, giving more complex instruction to big models, like Qwen 7B, and simple one to the small models. The evaluation is inspired by genetic algorithm, where a children (new generated prompt) substitute the father (which is the actual best prompt evaluated) only when its score a better accuracy on the training set. The last function is mutate prompt and it is the core of the project: we pass as input the prompt used, one of the wrong answer and the problem and then, in a two prompt pipeline, the prompt is first reviewed from a model that reason about the specific mistake and output a reason for the mistake: the answer is then passed in another prompt to the same model with the failed prompt and the reason that make the algorithm fail, and this information is used to execute a new prompt. The two phase-pipeline idea was inspired by the ProTeGi solution and aims to solve the not universality problem, where the model overfit too match on the error that was passed.

4.4 Evaluation phase on Colab

The run evolution function will return the original prompt and the best prompt that the algorithm produced, and then proceed to evaluate them in a 10 question tournament: for both the prompts, ten answer (equal for both) are drawn from the evaluation dataset and the student will solve the problems using both the prompts. At the end accuracies of each prompt are shown on a bar plot that measure the score for both the prompts. A success is obtained if in the training phase we obtain a new prompt, and if it obtains better performance also on the evaluation part.

5 Experiment result

5.1 1.8B Qwen model

The first tangible results where obtained with the Qwen 1.8B model, which was able to reason correctly and to give clear answer that could be interpreted by the judge model. Even if in many cases it failed on giving correct answer, a good prompt optimization could be done. On average it was good at following brief and not too complex instruction, and performance started to degradate if it had to carry out many instruction or if rule were given to it. Also command that ask him to not do something were ineffective, making him actually do the opposite.

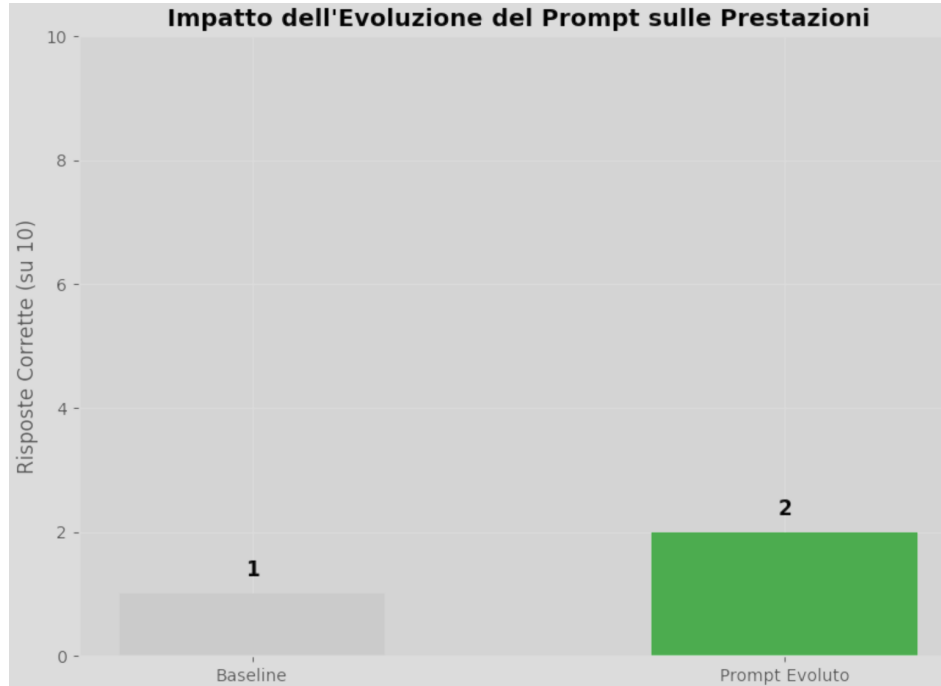


Figure 3: On the left the performance of the Qwen 7B model with the basic prompt, on the right the prompt that achieved the best results in the training.

5.2 7B Qwen model

The 7B model was very good at following instruction, and was able to order correctly many puzzle and to give step by step reasoning that lead to the correct answer. However sometimes the model just get confused during the reasoning and add nonsense noise to the text, only to get to the correct answer later. This prove the limit of the reasoning inside SLM. On a single test the final result are shown in the figure below:

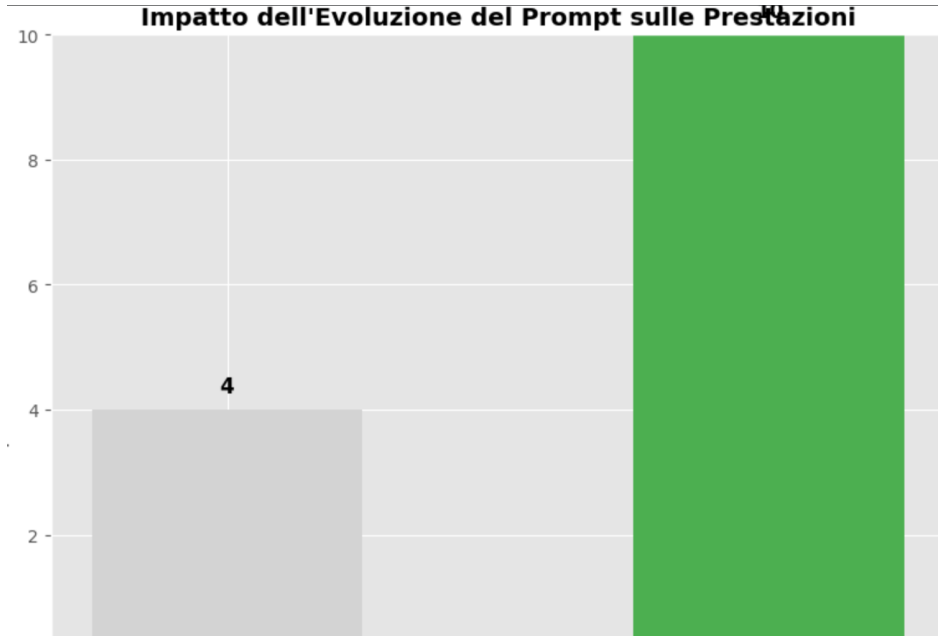


Figure 4: On the left the performance of the Qwen 7B model with the basic prompt, on the right the prompt that achieved the best results in the training.

This particular prompt evolution is actually very unlikely to happen, and depend from many variables. Also the judge can commit error in the evaluation and modify a little the structure.

5.3 Gemma as a judge and prompt evolver

Gemma perform very well as a prompt evolver and handle correctly the task, giving as output correct input for both the models, more simple for the 1.8B and more complex, requiring thinking step by step or with chain of thought for the 7B model. As said before the prompt evolution was done in two steps, inspired by ProTeGi solution from the paper Automatic Prompt Optimization with “Gradient Descent” and Beam Search. The solution is implemented in two fundamental steps: the first one is the abstraction of the failure, where the model receive the prompt that was used, the question where it failed and student answer. The abstraction is then used in the second step to create a new prompt, increasing the performance of the generation with respect to giving just a single prompt without pre-abstract the concept, who often carry the problem into the new prompt, overfitting it on the specific question. As a judge, while it behave better than Qwen, it still, even if rarely, fail to understand the reasoning of the student, leading to incorrect evaluation.

6 Limit of the models

During the experimentation i understood that i could not use the same model to be both the student and the evaluator at the same time without sacrifice some performance, especially on false positive case, where the student get the answer wrong but is still rewarded from the judge. The model is also limited by the training, which is very poor, but still require an extensive time (between 10 and 20 minutes on A100 to try 5 prompt with a batch size of 5), giving very poor visual of the behaviour of the prompt with respect to the true data distribution. The model training only on a few of examples,

in the end just get a prompt who can perform well on that little set, but in the reality, during training phase, it can produce worst outcome then the original prompt, as shown below.

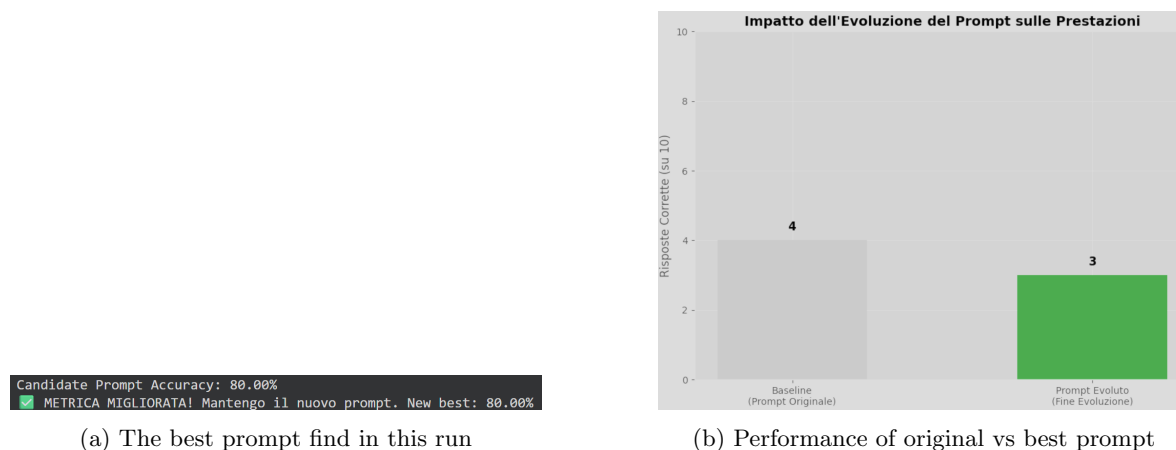


Figure 5: We can see that performance get worst with the new prompt

7 Possible improvement

While the experiment, for computational reason, was done only on a small data sample, it will be interesting to see how it perform with a bigger batch. Also, the prompt evolver is probably the bottle neck of this 3 model experiment, suffering a lack of creativity and producing very similar new prompts, lacking the capability of really grasp the cause of the failure and so of putting up a valid solution. For this it is possible to improve the automatic prompt evolver by adding a parallel exploration of multiple prompt and making it become a beam search, following the model proposed in ProTeGi, to better explore the space of the prompt solutions. Finally it is possible to move all the training phase on the prompt evolver module to have a more compact and readable code.

8 Bibliography

<https://arxiv.org/pdf/2305.03495>